



Fin Sight

PROJECT DOCUMENTATION

Personal Finance AI Tracker

Upload a bank statement — FinSight categorizes every transaction with a hybrid rules + LLM pipeline, charts your spending, and answers questions in plain English through validated text-to-SQL.

Stack FastAPI · SQLAlchemy 2.0 · PostgreSQL + MongoDB · pluggable LLM (Claude / Gemini / Groq / Ollama) · React + TypeScript + Recharts

Tests 78 backend tests, zero-infrastructure suite · **Author** Chitra Ameta · **Date** July 2026

Contents

1	Overview
2	Architecture
3	Data model
4	Feature deep-dives — auth, CSV import & dedup, categorization pipeline, insights, Ask AI
5	API reference
6	Configuration reference (incl. free LLM providers)
7	Frontend
8	Testing
9	Running the project
10	Troubleshooting
11	Known limitations & future work

1 • Overview

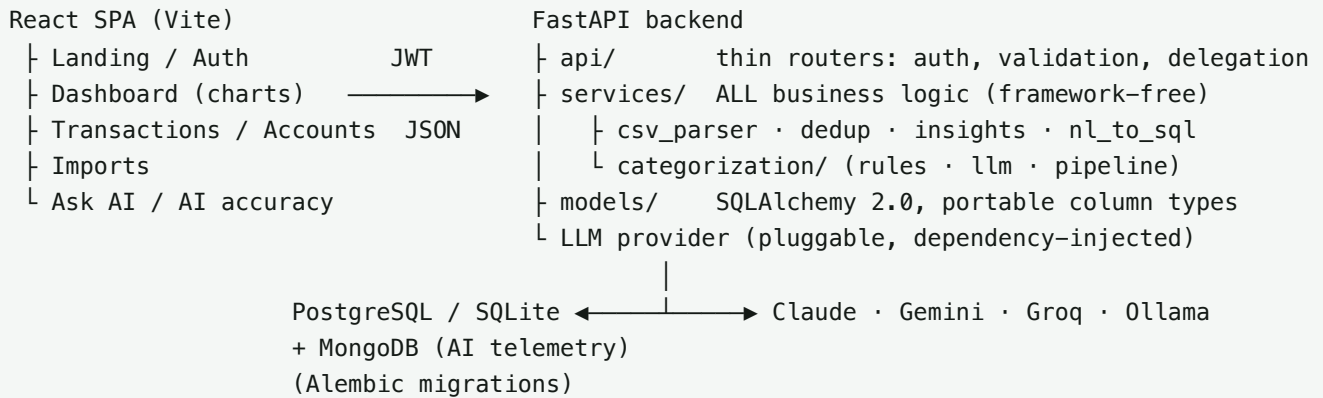
FinSight ingests raw bank-statement CSVs and turns them into:

- **Categorized transactions** — a hybrid keyword-rules + LLM pipeline;
- **Spending insights** — by category, by month, by merchant;
- **Plain-English answers** — “how much did I spend on food last month?” via LLM-generated, server-validated SQL.

It is a backend-first portfolio project. The priorities are clean architecture, API design, inspectable AI integration, and test coverage (**78 backend tests**) — not production hardening or UI breadth.

The two ideas that define the project: (1) every AI prediction — from rules *and* the LLM — is persisted to an audit table, so the two methods can be compared side by side; (2) AI-generated SQL is never trusted — it passes five validation layers before touching the database.

2 · Architecture



Layering rules

- **api/** routers stay thin — no business logic.
- **services/** hold all logic as plain functions/classes → unit-testable without HTTP.
- **models/** use deliberately portable column types (`Uuid` , string-backed enums, `JSON` , `Numeric`) so the same code runs on Postgres (dev/prod) and in-memory SQLite (tests).
- **LLM access** goes through `Protocol` interfaces injected as FastAPI dependencies — providers are swappable, tests inject fakes.

3 · Data model

`user_id` is denormalized onto `transactions` so every query — including AI-generated SQL — can scope with a plain `WHERE user_id = :id`. Money is always `NUMERIC(12,2)`, never float; the sign convention is **negative = money out**.

Table	Key columns	Purpose
<code>users</code>	email (unique), hashed_password, role (user/admin), is_active	Auth + RBAC
<code>accounts</code>	user_id, name, account_type, currency	Bank/card accounts
<code>categories</code>	name, user_id (NULL = system), unique (name, user_id)	15 seeded system categories + user custom
<code>import_batches</code>	account_id, filename, status, total/imported/duplicate rows, error	One row per CSV upload — import audit trail
<code>transactions</code>	txn_date, merchant_raw/normalized, amount, category_id, categorization_source, dedup_hash (unique per account)	The core ledger
<code>categorization_results</code>	transaction_id, method (rule/llm), predicted_category_id, confidence, detail JSON, latency_ms	Every prediction from both methods
<code>ai_events</code> MongoDB	kind, user_id, model, batch_size, latency_ms, status, free-form payload	One document per LLM call — polyglot persistence: relational ledger in Postgres, schema-flexible telemetry in Mongo
<code>nl_queries</code>	question, generated_sql, status (executed/rejected/failed), row_count, error, latency_ms	Audit log of every Ask-AI attempt

Migrations live in `backend/alembic/`; the initial migration seeds the 15 system categories (groceries, dining, rent, utilities, subscriptions, transportation, entertainment, shopping, health, travel, education, income, transfers, fees, other).

4 · Feature deep-dives

4.1 Authentication & RBAC

- bcrypt-hashed passwords (the `bcrypt` library directly; schema caps passwords at 72 chars — bcrypt’s hard input limit).
- OAuth2 password-form login → JWT with `sub`, `role`, `iat`, `exp`. Unknown email and wrong password return the same 401 so the API never leaks which emails exist.
- `get_current_user` + `require_role(...)` dependencies; foreign resources return 404, not 403 (existence isn’t leaked).

4.2 CSV import & deduplication

Parsing. Banks disagree on everything, so the parser normalizes: column names (case-insensitive detection of date / description / merchant, single `amount` or `debit / credit` pair), amounts (`$1,234.56` , accounting-style `(45.00)` negatives, optional `invert_amounts` for card exports), and six date formats. Bad rows are skipped and reported per line; a structurally unusable file fails the batch with a clear error.

Merchant normalization. `TRADER JOE'S #123` → `Trader Joe's` ; `AMZN Mktp US*2K4XY1` → `AMZN Mktp US` — powers merchant aggregation.

Dedup. No bank IDs exist in CSVs, so rows are fingerprinted: `sha256(date | amount | merchant | description | ordinal)` , where *ordinal* counts identical rows within one file. Re-uploads are fully idempotent; two identical same-day coffees both survive. Known limitation: the same purchase in two *overlapping* exports reads as a duplicate. Dedup is scoped per account.

4.3 Hybrid categorization pipeline

```
transaction → rule classifier — confidence ≥ 0.7 → categorized (source=rule)
                |
                | below threshold → batched to LLM (~20 per call)
                |                                     |
                |                                     | ok → categorized (source=llm)
                |                                     | LLM down ► weak-rule fallback, else
                |
uncategorized
every prediction (rule AND llm) → categorization_results ← the audit trail
```

- **Rules:** ~80 keyword rules, each with its own confidence. Specific merchants score high (`netflix` → subscriptions 0.95); ambiguous ones are deliberately under-threshold (`venmo` 0.6, `paypal` 0.5, `amazon` 0.75) to route to the LLM. Income keywords are sign-gated (“PAYROLL” on money-out is discounted). Highest confidence wins (resolves `uber eats` vs `uber`).
- **LLM:** batches of ~20 per call — a ~20x cost/latency win. Output is schema-validated; off-list categories are rejected; confidences clamped to [0, 1]. Model id, prompt version, batch size, latency recorded.
- **Degradation:** an LLM failure never fails the run — affected transactions use their weak rule match and the error surfaces in the run summary.
- **Comparison:** because both predictions persist, the API serves per-transaction history and aggregate agreement stats — the “AI accuracy” page.

4.4 Insights

Three dialect-portable aggregates (SQLAlchemy `extract()` compiles to `strftime` on SQLite, date parts on Postgres): **spend by category** (spending only, explicit `uncategorized` bucket, date filters), **monthly summary** (spent / income / net per calendar month), and **top merchants** (grouped on normalized merchant). Sums are normalized to 2-decimal `Decimal`s; the API serializes money as strings.

4.5 Ask AI — natural language to SQL

Design principle: the LLM only *proposes* SQL; nothing it produces is trusted.

Five layers stand between the model and the database:

- **1 · Parse** with sqlglot — unparseable input rejected.
- **2 · Statement check** — exactly one plain SELECT (no unions, DML, DDL; `SELECT 1; DROP TABLE users` dies here).
- **3 · Table allowlist** — only transactions / categories / accounts, unqualified; CTE names shadowing real tables are rejected.
- **4 · Server-side scoping** — every table reference is rewritten to a `scoped_*` CTE pre-filtered to `:user_id`, so even a query with no user filter can only see the caller's rows.
- **5 · Row cap** — LIMIT 200 injected or tightened.

Every attempt is audited in `nl_queries` as executed / rejected (offending SQL preserved) / failed. The generator prompt describes the schema as already user-filtered, explains the sign convention, lists category names, injects today's date so "last month" becomes explicit ISO literals, and hints that "food" = groceries + dining.

5 • API reference

Interactive Swagger docs at </docs>. All endpoints except register / login / health require **Authorization:** Bearer <token>.

Endpoint	Body / params	Returns
POST /auth/register	{email, password} (8–72 chars)	201 user · 409 duplicate email
POST /auth/login	form: username, password	{access_token, token_type}
GET /auth/me	—	current user
GET /auth/users	—	all users — admin only
POST /accounts · GET /accounts	{name, account_type, ...}	own accounts
POST /uploads	multipart: file, account_id, invert_amounts?	{status, total/imported/duplicate_rows, error}
GET /uploads	—	last 50 batches
GET /transactions	account_id?, limit ≤ 500, offset	transactions, date-desc
GET /transactions/{id}/categorization	—	rule + LLM prediction history
POST /categorization/run	account_id?	{total, rule_categorized, llm_categorized, still_uncategorized, llm_error}
GET /categorization/stats	—	{by_source, compared, agreements, agreement_rate}
GET /categories	—	system + own categories
GET /insights/by-category	start?, end?	[{category, total_spent, transaction_count}]
GET /insights/monthly	months (1–60)	[{month, spent, income, net}]
GET /insights/top-merchants	limit ≤ 50, start?, end?	[{merchant, total_spent, transaction_count}]
GET /ai/events	limit ≤ 200	{enabled, events[]} — raw LLM call telemetry from MongoDB
POST /nlq	{question} (3–500 chars)	{sql, columns, rows, row_count, latency_ms} · 400 rejected/failed · 502 generation error · 503 no LLM
GET /nlq/history	—	last 50 attempts with status + errors

```
TOKEN=$(curl -s -X POST :8000/auth/login -d 'username=you@x.com&password=...' | jq -r
.access_token)
curl -s -X POST :8000/nlq -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"question": "how much did I spend on food last month?"}'
```


6 · Configuration reference

All settings load from `backend/.env` (template: `backend/.env.example`).

Variable	Default	Purpose
<code>MONGODB_URL / MONGODB_DB</code>	— / <code>finsight</code>	MongoDB for AI telemetry; optional — unreachable server disables it gracefully
<code>DATABASE_URL</code>	local Postgres	SQLAlchemy URL; <code>sqlite:///./dev.db</code> works for local dev
<code>JWT_SECRET_KEY</code>	change-me...	JWT signing key — set a real one
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	1440	Token lifetime
<code>LLM_PROVIDER</code>	anthropic	<code>anthropic</code> or <code>openai</code> (any OpenAI-compatible endpoint)
<code>ANTHROPIC_API_KEY / _MODEL</code>	— / <code>claude-haiku-4-5</code>	Provider = anthropic
<code>OPENAI_API_KEY / _BASE_URL / _MODEL</code>	— / <code>api.openai.com / gpt-4o-mini</code>	Provider = openai
<code>LLM_BATCH_SIZE</code>	20	Transactions per categorization call
<code>RULE_CONFIDENCE_THRESHOLD</code>	0.7	Below this, transactions go to the LLM

Free LLM setups (all with `LLM_PROVIDER=openai`)

Provider	OPENAI_BASE_URL	OPENAI_MODEL	Key from
Google Gemini (free tier)	<code>https://generativelanguage.googleapis.com/v1beta/openai/</code>	<code>gemini-2.5-flash</code>	<code>aistudio.google.com</code>
Groq (free tier)	<code>https://api.groq.com/openai/v1</code>	<code>llama-3.3-70b-versatile</code>	<code>console.groq.com</code>
Ollama (local, keyless)	<code>http://localhost:11434/v1</code>	<code>llama3.2</code>	—

With no key configured, categorization runs rules-only and Ask AI returns a clear 503 — everything else works.

7 • Frontend

React 18 + TypeScript (strict) + Vite + Recharts. Brand: **FinSight** — deep emerald + warm gold, full dark mode via `prefers-color-scheme`. Chart series colors use a validated colorblind-safe palette kept separate from brand chrome. API base URL comes from `VITE_API_URL` (default `http://localhost:8000`).

Piece	File	Notes
API client	src/api.ts	Typed endpoints, JWT in localStorage, auto-logout on 401
Landing	components/Landing.tsx	Hero with CSS-only dashboard preview, features, steps
Auth	components/Login.tsx	Split screen: emerald brand panel + form
Shell	components/Shell.tsx	Sidebar (Overview / AI tools / Data), user card
Dashboard	pages/DashboardPage.tsx	Welcome banner, 4 accented KPI tiles, trend, donut, merchants
Transactions	pages/TransactionsPage.tsx	Account filter + live search; decision-source pill per row
Accounts / Imports	pages/...	Account CRUD · upload widget + batch history
Ask AI	pages/AskPage.tsx	Question box, suggestions, results, generated SQL, history
AI accuracy	pages/AccuracyPage.tsx	Agreement rate, who-decided bars, pipeline explainer

Chart discipline: donut capped at 6 segments (top 5 + “Other”), single-hue ranked merchant bars, 2px strokes, hairline grids, legends + tooltips, dark palette selected for the dark surface (not auto-inverted).

8 • Testing

```
cd backend && pytest          # 78 tests, ~15s, no services needed
```

- Tests run on **in-memory SQLite** — the reason model column types are portable.
- The app is tested through `TestClient` with `get_db` overridden per test; each test gets a fresh seeded database.
- **LLM calls are never made in tests** — classifier and SQL generator are dependency-injected and replaced with fakes; the OpenAI-compatible provider is tested against a stubbed client.

Area	File	Tests
Models & constraints	test_models.py	3
Auth + RBAC	test_auth.py	11
CSV parsing + dedup	test_csv_parser.py	11
Uploads / idempotency	test_uploads.py	8
Categorization rules / pipeline / API	test_categorization.py	13
Insights	test_insights.py	7
NL-to-SQL validation + API	test_nlp.py	17
OpenAI-compatible provider	test_openai_compat.py	4
Mongo telemetry (fake collection)	test_ai_telemetry.py	4

9 · Running the project

Standard (Postgres via Docker)

```
cd backend
docker compose up -d
python3 -m venv .venv && source .venv/bin/activate    # Python 3.11+
pip install -r requirements.txt
cp .env.example .env                                # fill in secrets
alembic upgrade head
uvicorn app.main:app --reload                        # API :8000, docs at /docs

cd ../frontend
npm install
npm run dev                                           # UI :5173
```

No Docker (SQLite)

Set `DATABASE_URL=sqlite:///./dev.db` in `.env`, then `alembic upgrade head` and run uvicorn as above.

Different API port

If `:8000` is taken: `uvicorn app.main:app --port 8010` and `VITE_API_URL=http://127.0.0.1:8010 npm run dev`.

10 • Troubleshooting

Symptom	Cause / fix
“address already in use” on :8000	Another service owns the port — run on <code>--port 8010</code> and set <code>VITE_API_URL</code>
UI: “Could not reach the API”	Backend not running, or <code>VITE_API_URL</code> mismatch; CORS allows localhost:5173 only
Ask AI returns 503	No LLM key configured — see §6
Ask AI returns 400 “Query rejected”	Working as intended — validation refused the generated SQL; see <code>GET /nlq/history</code>
Gemini 429 “limit: 0”	Free tier doesn’t cover that model id — use <code>gemini-2.5-flash</code> or <code>-lite</code>
Run summary shows <code>llm_error</code>	Provider outage / bad key; affected rows used weak-rule fallback — re-run later
Upload says all rows duplicate	Same statement re-uploaded — idempotency working
<code>str None</code> TypeError on startup	Python < 3.10 — use 3.11+ (project venv uses 3.12)

11 · Known limitations & future work

- **Dedup across overlapping exports** — without bank transaction IDs, the same purchase in two overlapping statements reads as a duplicate.
- **Read-only DB role for NL queries** — validation enforces SELECT-only at the parser level; production would additionally execute NL queries on a read-only Postgres role with a statement timeout.
- **Manual re-categorization UI** — the `manual` source exists in the schema; no edit-category UI yet.
- **Natural extensions** — recurring-subscription detection, monthly budgets with alerts, Plaid bank-link ingestion, CSV/OFX export.

FinSight · Personal Finance AI Tracker — project documentation, July 2026.